Daily Log

This is where I'll record hopefully before class what we will be doing. Please check here and read before class, so you'll have an idea of what we will cover that day.

---

day 21 | 251017 F

We are going to make a jump to solving differential equations. We are going to start with the simplest method of solving differential equations, Euler's method, and we are going to do it with excel or google sheets to get a sense of what is going on, then we will learn how to code it up in python. But seeing it in excel is a great way to start off.

Euler's Method is all about predicting the value of the function by knowing the value of the derivative and a time interval that has gone by.

---

day 20 | 251015 W

We want to practice using several methods together, and although I messed this up somewhat I'll show you how to do a better problem at the end. But here is the problem at the moment that I would like to solve.

We have the Stephan-Boltzmann Law that says

$$W = \sigma T^4$$
And what I would like to know is how to solve for T, given that we know a value for W. So we can make W=20, and then apply some method to finding the root of the resulting equation, which is like asking for which value of T is the following function 0.

$$f(T) = \sigma T^4 - 20 => 0$$
So that is the function that we want to put into our secant method or bisection methods or whichever method you want.

First you should plot it. That will give you some idea of where the zero is. Then use bisection or my favorite secant method to find the temperature T that will make this happen. It is easy enough to check this with algebra, so for our example

$$T = \sqrt[4]{\frac{20}{\sigma}} = \sqrt[4]{\frac{20}{5.652\times10^{-8}}} = 137.15$$

---

day 19 | 251010 F

Today, we continue to look into root finding, this time reviewing several techniques. I'll refer to the code at the bottom of this section.

The bisection method is very simple to think through. Take two guesses that are on either side of the root. Find the midpoint between these guesses, and then decide whether that midpoint should be the new upper bound or lower bound. That decision is then repeated over and over until you reach a *desired tolerance*. This method is easy and works well as long as you made decent guesses of upper and lower bound. You should probably graph the function first to determine these, since it can be difficult to do without a good guess.

Another clever way to do this is to use a line that is fit to an initial guess. This is known as Newton's Method, and in order to do this, we need to know the point as well as the derivative at that point. This is easy for many functions that we can write down algebraically but not all functions. But it works very quickly and is easy to understand.

The final method is a very good one, because we only need to provide one guess and we don't need to use or know the derivative function. We evaluate the function at two different places that are "near" by and then fit a line to that. The method itself works similarly to the Newton's method from there.

Homework is to do exercise 6.16, specifically part (B).

```
def bisection(function, lower_guess, upper_guess,
tolerance=2**-32):
    midpoint = (lower_guess + upper_guess)/2
    while upper_guess - lower_guess > tolerance:
        if function(lower_guess)*function(midpoint)<0:
            upper_guess = midpoint
            midpoint = (lower_guess + upper_guess)/2
        elif function(midpoint)*function(upper_guess)<0:
            lower_guess = midpoint
            midpoint = (lower_guess + upper_guess)/2
        elif function(lower_guess)*function(midpoint)>0 and
function(midpoint)*function(upper_guess)>0:
            print('no unique root in that bracket')
            break
    return(midpoint)

def newton(f, df, guess, tolerance = 2**-32):
    x = guess
    n = 0
    while abs(f(x)) > tolerance:
        x = x - f(x)/df(x)
        n += 1
    return(x, n)
```

```
def secant(f, guess, delta, tolerance = 2**-32):
    x0 = guess
    x1 = x0 + delta
    n = 0
    while abs(f(x1))>tolerance:
        x1 = x1 - (x1-x0)/(f(x1)-f(x0))*f(x1)
        n += 1
    return(x1, n)
```

---

day 18 | 251008 W

We began a series on root finding. What we mean by root finding is finding where (as in what x-value) a function is equal to zero. We can use these techniques to find the value of a function when it does not cleanly work out in the data that we have generated. It may in fact be difficult to interrogate a function backward like this. So we can employ several methods which we will look at in the coming days to help investigate this problem.

The first of these methods is the *relaxation method*. This method involves us making a guess and evaluating a function. So as an example we can use

$$f(x) = x-e^{-x}$$
There is no algebraic way to find a value for x that results in the output of this function being equal to zero. But we can take this equation and rearrange it somewhat:

$$ x = e^{-x} $$
Now we are going to look for values of x that give the same value on both sides of this equation. One very clever hack for this method is to simply set the value and reuse that on the other side of the equation. This is the method at the heart of the relaxation method. I'll make a guess, evaluate (e^-x)), then set that value no matter what it is equal to x, and then continue to re-evaluate (e^-x)). This will *slowly* creep up to the value that I am looking for. Below is the code that we will use to do this:

```
x = 2 # initial guess
error = 1 # initial error
count = 0 # counting index to see how many steps
while abs(error)>0.00001:
    x1 = np.exp(-x)
    error = x1 - x
    x = x1
    count = count + 1
    if count > 100000:
        print('i couldnt find an answer so ignore this')
        break
```

```
print(x)
print(count)
```

---

day 17 | 251006 M

We took a day to work on homework assignments that are long over due. We worked together and debugged some code in class today.

---

day 16 | 251003 F

We will look at the question of whether decreasing the step size of a derivative always increases the accuracy (spoiler: it doesn't, and its complicated).

We will also look into methods for differentiating DATA, which is similar and easy, but there are some "gotchas" there as well that we need to be aware of.

Work exercise 5.15.

---

day 15 | 251001 W

We need to clean up some things from last time (see my email about this to you all). Then we will turn to differentiation, while we also work on some above average homework problems. These are exercises 5.9 and 5.12.

We have been talking about how to do calculus, in particular the integral up till now. But how do we do the derivative? First of all, the calculus itself is nothing other than operations to find either the slope of a function (derivative) or the area under the curve of a function (integral). We often times lose track of what we are actually trying to do when we do this algebraically, so I think it is important to encounter the computational side of things to more fully grasp what is going on.

First off, numerically doing derivatives is very simple, but it is not often done using computers for this reason. If you can do them in your head then why use a computer? But, this is not true for integrals! There are many integrals that do not have a closed form, algebraic solution; so computational methods are necessary to evaluate them. There are also problems with calculating derivatives as we will see, and so they are less used.

But, first of all we have the definition of the derivative from calculus class $$\frac{df}{dx} = \lim_{h \rightarrow 0}\frac{f(x+h)-f(x)}{h}$$ And we can do this, just fine except

that we cannot divide by zero. But we can make `h` small and maybe that is good enough. $$\frac{df}{dx} \approx \frac{f(x+h)-f(x)}{h}$$ This is an approximation that is known as the *forward difference*. And so naturally there is such a thing as the *backward difference*: $$\frac{df}{dx} \approx \frac{f(x) - f(x-h)}{h}$$

Unfortunately, both of these methods have significant errors. Fortunately, they miss the true value in opposite directions. So we can simply *average* them and get a great approximation. This is known as the *central difference* or *mean finite difference* operator: $$\frac{df}{dx} \approx \frac{f(x+h)-f(x-h)}{2 h}$$

If you need an even better method, then the symmetric four-point method is the one you want: $$\frac{df}{dx} \approx \frac{f(x+2h)-8f(x+h)+8f(x-h)-f(x-2h)}{12h}$$

Now what do we choose for `h`? We already know that the smallest the computer can go is machine epsilon. But does that mean it is the best number to use? Let's take a simple function like $$ f(x) = \frac{1}{3}x^3$$ and evaluate the derivative of this function at x=1. And then we will compare the results to those we can do by hand.

Second derivatives are easy to derive from first derivatives. Using the central difference method: $$\frac{d^2f}{dx^2} \approx \frac{f'(x+h/2)-f'(x-h/2)}{h}$$ $$\frac{d^2f}{dx^2} \approx \frac{[f(x+h)-f(x)]/h - [f(x)-f(x-h)]/h}{h}$$ $$\frac{d^2f}{dx^2} \approx \frac{f(x+h)-2f(x) +f(x-h)}{h^2}$$

So implementing a double derivative is not that much more difficult than a single derivative.

```
def back(function,x_value, stepSize):
    return (function(x_value)-function(x_value-
stepSize))/stepSize

def forward(function,x_value, stepSize):
    return (function(x_value+stepSize)-
function(x_value))/stepSize

def mid(function,x_value, stepSize):
    return (function(x_value+stepSize)-function(x_value-
stepSize))/(2*stepSize)

def doubled(function, x_value, stepSize):
    return
(function(x_value+stepSize)-2*function(x_value)+function(x_value-
stepSize))/(stepSize**2)
```

---

day 14 | 250929 M

We will discuss some other integration methods. We will talk though some quadritures (which is an old word for numerical integration). I have some defined below, but we will also use a function that is provided to us by the author, namely `gaussxy.py` which has a couple of methods within it that we will use. It is important to note that you need to have this python script put in the same folder that you are going to call it from, so whether that is here in `develop` for today, or a copy in `deliver` for any homework, you'll need it in both places. Also, as I mentioned in the email I sent you, this particular bit of script has been updated by the author, so the version that you have in your `data` folder will only work if you look at it and make it work for you.

We also looked at integrals over infinite ranges, and we used a change in variables to handle those situations. In most cases you can make the following substitution:

$$z = \frac{x}{1+x} \quad \longleftrightarrow \quad x = \frac{z}{1-z}$$
There are other variations on this that you should be aware of like

$$z = \frac{x-a}{1+x-a}$$
but in either case

$$dx=\frac{dz}{(1-z)^2}$$
Now you can integrate either from 0 -> ∞ or a -> ∞. To do the other side like from $-\inf$ to 0 or to a, then make z -> -z up above.

---

day 13 | 250926 F

These days got away from me with the notes. I think we worked on some homework problems as well as integrating a set of data rather than a function. This point is worth emphasizing. When you have data provided to you, or produced from a machine and recorded, there are limitations to how many points you can use to integrate up. So the trapezoid method is likely the best case since it is so simple to implement and use. Simpson's method is trickier because you have to have an odd number of points. So you can use Simpson's method for the odd number and then trapezoid at the end for the last point if you want, and that is what many codes do. Here is one that I cooked up last spring that beat scipy's integrate function in a contest that i am very proud of:

```
def integrate(x, y):
    '''simpson's rule for integration with simpson’s 3/8 rule
adjustment for even-length data'''

    if isinstance(x, pd.Series):
        x = x.to_numpy()
        y = y.to_numpy()

    steps = len(y)
```

```
    h = (x[-1] - x[0]) / (steps - 1)  # step size (ensuring
correct intervals)

    if steps % 2 == 1:  # odd number of points -> use
standard Simpson's 1/3 rule
        s = (h / 3) * (y[0] + y[-1] + 4 * np.sum(y[1:steps-
1:2]) + 2 * np.sum(y[2:steps-2:2]))

    else:  # even number of points -> use Simpson’s 1/3 rule
for first n-3 and 3/8 rule for last three
        s_simpson = (h / 3) * (y[0] + y[-3] + 4 *
np.sum(y[1:steps-3:2]) + 2 * np.sum(y[2:steps-4:2]))
        s_three_eighth = (3 * h / 8) * (y[-3] + 3 * y[-2] + 3
* y[-1] + y[-1])  # last 3 points
        s = s_simpson + s_three_eighth

    return s
```

See if you can improve it!

---

day 12 | 250924 W

I have no idea do you?

Katie Marie says refer to day 11....

---

day 11 | 250922 M

Today we will continue with integration, but we will flip the script and you will work together to implement Simpson's Rule and do exercises 5.2 and 5.3. Simpson's Rule is based on fitting a parabola to a portion of the curve we are interested in and then iterating over each parabola. So look at equation 5.9 on page 146 in the textbook and figure out how to pull that off in python using the numpy.sum function.

Also while we are here I want to emphasize something to you. Look at the author's comment after the problem on page 148. He says, "Note that there is no known way to perform this particular integral analytically, so numerical approaches are the only way forward." This is a very important point and one worth keeping in mind as we go through this class. We are doing things that are increasingly impossible to be done any other way. So computational physics is often not a short cut or a way out of certain problems, but rather it is the only way to get a particular answer.

---

day 10 | 250919 F

Integration is essentially addition and so there are fewer problems with dividing by a small number like we had with differentiation. But we still need to be careful about errors like round off. Also similar to differentiation, there is a forward and backward way to integrate, and neither of these are accurate enough to be useful, but there is something in between, in the case of integration that is the trapezoidal rule. And like with differentiation there will be a slightly more complicated method, but also even better than trapezoidal and that is Simpson's rule.

We will start with Riemann's definition of the integral:

$$\int^b_a f(x) \mathrm{d}x = \lim_{N\rightarrow \infty} \sum_{n=0}^{N-1} f(x_n) h = \lim_{N\rightarrow \infty} \sum_{n=1}^{N} f(x_n) h$$
This simply tells us that these right and left sums are the same when the divisions go to infinity. But of course we can't go to infinity. So, a trapezoidal rule is an average of both the right and left rectangle rules is a way of averaging both:

$$\int^b_a f(x) \mathrm{d}x \approx \sum_{n=0}^{N-1} \frac{f(x_{n+1}) +f(x_n)}{2}h$$
This has the effect after factoring out h/2 and expanding the sum of: $$\int^b_a f(x) \mathrm{d}x \approx \frac{h}{2}[f(x_0) + 2f(x_1) + 2f(x_3)+...+ 2f(x_{N-1}) + f(x_{N})]$$

Which can be rewritten as $$\int^b_a f(x) \mathrm{d}x \approx \frac{h}{2}[f(a) + f(b)] + \sum_{n=1}^{N-1}f(x_n)h$$

And now we can calculate this sum much faster. Implementing this can have some "gotchas", so we have to be careful about how we go about pulling this calculation off.

Do exercise 5.1 to practice this. But also go back and work on exercise 3.8 as a call back to that chapter on graphing. This is a classic example of a problem that *seems* very difficult, but actually the author walks you through nicely.

---

day 9 | 250917 W

We used most of our time to talk about homework questions and clean up some previous comments. We also talked a good bit about calculus and specifically integration which we will do a lot of moving forward. We talked about Reimann sums and began to cook up a function that would do this for us, but ran out of time.

---

day 8 | 250915 M

Today we will practice plotting with real data that would be gathered from some instrument and which you want to plot for your scientific notebook or for a paper. Now that we have the basics of plotting we need to practice this with some real data and see what trouble we run into. So today we will import and trim data, and then prepare some plots based on that data.

To import some data to python, there are a variety of ways, each depending on how complex the data is. Today, we will start with a simple example of two columns of data in a file. The file extension is readable by python as simply a `comma separated value` or `csv` file. Many files that have various extensions are like this, and many/most experimental apparatus will have the ability to export data in such a format.

After loading the data using `np.loadtxt()`, we have to take the transform of this file using `data.T`, and then we are able to use matplotlib to plot the data just like before.

Pandas has a more complicated but more capable function called `pd.read_csv()`. This is our first encounter with data stored as a pandas `dataframe` object, but we will practice how to use it to plot some data.

---

day 7 | 250912 F

Now that we have the basics of plotting we need to practice this with some real data and see what trouble we run into. So today we will import and trim data, and then prepare some plots based on that data. We also want to check in and follow up on the last few assignments that you have, namely finishing printing out Pascal's triangle as well as finding a python graphing module and showing that off to the class.

To import some data to python, there are a variety of ways, each depending on how complex the data is. Today, we will start with a simple example of two columns of data in a file. The file extension is readable by python as simply a comma separated value or csv file. Many files that have various extensions are like this, and many/most experimental apparatus will have the ability to export data in such a format.

After loading the data using np.loadtxt(), we have to take the transform of this file using data.T, and then we are able to use matplotlib to plot the data just like before.

Next, we will choose a more complicated file, and talk through how we can import it using pandas. Pandas is a python library specifically for handling more complicated data files than numpy is good for. Notice for instance that pandas handles text headers to give a column a particular name. Each pandas column acts like an individual numpy array.

In this case we have a file that has a large header that we need to deal with, as well as a format problem. We will use a statement like this to load the data into what is called a "Data Frame", which is really just python's word for a database.

```
data = pd.read_csv('filename.txt', header=55, sep='\t',
encoding='windows-1252')
```

Now some of this is specific to this particular file, but these are some of the options you may need to use from time to time. Each column in this data file already has a name

This will lead us into a discussion of formats and what you can do with them. For example did you know that you can unzip a word document?

From here, work exercises 3.1 and 3.2.

---

day 6 | 250910 W

Pascal's Triangle

First we will work on Pascal's triangle, since I realized that you don't know about nested loops yet. So we'll get that to work and talk about those.

A nested loop always has a structure like this:

```
for i in some_kind_of_range:
    for j in some_other_range_or_list:
        do_some_stuff_with_i_and_j
    do_more_stuff_with_i
```

What this does is loop through one list and at each step loop through another list. These can be related to each other as in our example in class, or can be totally separate.

Chapter 3: plotting!

We will also begin working on plotting things. First we will just cook up some "data" by which I mean we will plot a particular function, and then we will look at some options around how to make this look nice. There are MANY options on how to plot things and change different settings, but we will stick to the simplest ones. Here is an example of how to plot:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.linspace(0,10)
y = x**2

fig0 = plt.figure()
ax0 = fig0.add_subplot()

ax0.plot(x, y, 'o', mfc='none')
```

This will plot $y=x^2$ from 0 to 10. Notice how I put in the option `'0'` and `mfc='none'` because that is the kind of plot that *I* like.

---

day 5 | 250908 M

Now that we have talked through the major operators in python, we need to put this altogether with the python function. Functions in python are small packages of code that can take in input and execute a series of steps based on that input, and optionally return an output. Functions are defined beginning with def statements so something like this:

```
def myFunction(inputs, go, here, if, necessary):
    print('this is a function')
    output = inputs*go*here**(if+necessary)
    return(output)
```

Now, we can write functions that can perform a particular calculation with different inputs, and we can begin to keep some of these so we can refer to them again later.

We wrote a couple of functions in class, one for calculating factorials and another for understanding spherical coordinates. For homework do exercise 2.11 and take an honest crack at 2.12.

---

day 4

First here is a Markdown Cheat sheet: https://github.com/adam-p/markdown-here/wiki/Markdown-Cheatsheet

Today we need to get to our first new kind of operations in python, `if`, `while`, and `for`.

`if` is a conditional statement, that is if some kind of condition is met, then it will execute a block of code that is underneath it. If that condition is not met, then it won't. You can see how I use if/then statements in that last sentence, and that is basically how

python works too. The `then` part of this command is what is executed inside of the `if` statement. IF that condition is not met, then a couple of things can be coded in to happening. Either nothing happens and the program continues with the next command that is issued, or you can put a `else` or `elif` commands and issue another set of instructions for the program to follow in the event that the `if` statement is not true. This true/false property gives us yet another data structure that we have not seen yet: the Boolean. It does not come up as a data type in its own right very often, but it is necessary to control `for` or `while` statements.

Similar to a `while` statement is a `for` loop. A `for` loop iterates over an input, and performs a function that is included within it. `for` and `while` loops can often be used interchangeably, but I find myself usually going to a `for` loop more often. The iterator can be many things, but we will start with base python `range` function and then also use numpy's functions `np.arange` and `np.linspace`. Much of what we do will be using `for` loops although there are some faster ways of doing things using a numpy array that will speed things up.

As an example, in class we wrote a `while` statement that printed the Fibonacci numbers up to 1000.

For homework, I want you to write a program that performs a sum of reciprocals. In mathematical language this would look like $s(k) = \sum_1^{100} \frac{1}{k}$. As a check on this, you should get about 5.187. I would use a `for` loop on this.

---

day 3

Today we will finish our discussion of data types, by talking about strings and talking about lists or arrays. We have already used strings to print statements in words to the terminal. They are an important class of data, but less used in physics simulations. However the most important data structure to us is the list, and the important upgrade to the array. We will learn to create and import a list, how to print it and will begin to see how to graph it.

A string is any kind of data stored between single quotes, such as `'Hello World'`. A string can contain numbers, but they are not stored or used as numbers but rather essentially as letters. If a number happens to be stored as a string and you would like to use it as a number, then you can use the `float` or `int` functions for this. There are a few other properties of strings that we can talk about once we get through the next section on *lists*.

A list is a collection of data structures, and while technically anything can go in the container, we will be dealing with lists mostly as a collection of float numbers.

---

day 2

Now that we have `conda` installed and working, we need to make sure it has all of the libraries that we will use. We will use matplotlib, numpy, pandas, and scipy for the most part although there may be others that would be nice. Let's try to use them first and then we will install them if necessary. To import these, we use an import statement like `import numpy as np`. This will bring all of the functions from the `numpy` package into our code, and then we can use them if we first use the `np` identifier. So for example we could now use `np.pi` to use numpy's value for pi. Keep in mind that this is slightly different from the way your book uses import statements. Our way might technically be slower overall, but we won't notice and this will allow us to explore things more easily.

We used a variable in Day 2, but a variable is simply a name or letter that stores a reference to another object in python. I have to *declare* a variable in order to use it, and in python that means I have to give it a value (even if it won't keep that value). `x` vs `x=5` vs `x=y`.

Also, python is very limited in the math that it can do for you. `x = 5*10` works just fine but `x=y/2` does not nor can python solve for another variable like `y`.

But something like `x = x + 1` works just fine (as long as x has been defined already), even though it makes no sense to us. In fact, we will do operations like this all the time!

Let's do a few examples (and we'll cover comments along the way):

1. a ball dropped from a tower, given a height and time, tell where the ball is.
2. conversion from polar coordinates to cartesian

day 1

Quickly review what we talked about last time.

1. Powershell/Terminal, cd ~, ls, pwd, mkdir
2. Opening a jupyter lab instance in a directory where you want to store things

What we want to talk about today are the various ways that you can run a script with python. So we are going to look at

1. The interpretor
2. A python script
3. A jupyter lab/notebook session

We will mostly be using jupyter lab sessions as they are interactive and offer a lot of opportunity for experimentation.

We will also show along the way how variables work and how basic math operations work. Everything is mostly what you expect with the exception of exponents, where 2^5 would be written as 2**5 using python. We will also cover code comments along the way.

day 0

This is where I have to tell you what you should install. Technically, I hope that everyone could install the following software on their computer. These are listed in order of priority

1. Anaconda distribution of Python
2. Mathematica
3. Dropbox (create account, but also download and install)
4. Keepassxc (not essential but a good habit)
5. git (extra special sauce)

If you could begin an account with the following:

1. Dropbox (from above)
2. Overleaf (free, student edition)
3. Github (extra if you get git working)

Talk about AI. I want students to learn to use it as much as they learn to use the computer itself. But start off without it. If you do use it, but big, bright lines around what you got it to teach you.

I talked through the python versions of 1. PowerShell/Terminal 2. Script 3. Jupyter Notebook/Lab. We did a very simple 'hello world' as well as a for loop just to show the differences in each, and went over some typo level gotcha's that come up. End on Jupyter Lab and showed them how that is a nice mid point between the interpreter and a script.

For the interpreter, you should just use terminal on Mac/Linux, and use Powershell for Anaconda on Windows.

For the script, I talk through using a shebang like #!/usr/bin/env python. We looked at mistakes and I showed them how to use the Traceback to kind of find where the problem is, although this is not perfect.

For jupyter, we will but using lab not notebook but they are very similar and students should know how to use both.

Here is chapter 2 of the book: chapter 2

In [1]:
```
!jupyter nbconvert --output-dir='.' --output="README.md" --to markdown --Tem
```

```
[NbConvertApp] Converting notebook Daily_Log.ipynb to markdown
[NbConvertApp] Writing 16119 bytes to README.md
```

In [ ]:
```
```